

CS3006 — Parallel and Distributed Computing

Assignment 2: Performance Analysis on a Multi-Core CPU

Muhammad Auon Naseer

Roll Number: 23L-0643 | Section: BCS-7A

FAST-NUCES Lahore | Fall 2026

1. Machine Declaration

Field	Value
CPU model	Intel Core i7-8565U @ 1.80 GHz
Physical cores (as exposed to guest)	8
Hardware threads (logical CPUs)	8
SMT / Hyper-Threading present?	Not distinguishable in guest (VirtualBox limitation); host CPU has 4 cores/8 threads via HT
Base / max clock	1.80 GHz base
Widest SIMD available	AVX2 (8-wide single precision)
RAM size and type	16 GB DDR4-2400, dual-channel, SODIMM
Machine type	VirtualBox VM (own laptop), Windows 11 host
OS, kernel, GCC version	Ubuntu 24.04, kernel 7.0.0-30-generic, GCC 13.3.0
ISPC version	1.31.0
Power state during measurement	Plugged in throughout

Note on hardware topology

This assignment was run on a VirtualBox VM. The VM was initially configured with 4 vCPUs (matching the physical core count of the host's i7-8565U) and was later increased to 8 vCPUs to match the host's full logical thread count (4 cores $\times$ 2 threads via Hyper-Threading), for closer fidelity to the addendum's target topology (Section 8). VirtualBox exposes these 8 vCPUs to the guest as 8 flat cores (Thread(s) per core: 1 in lscpu) rather than reproducing the host's 4c/8t sibling-thread structure. $T = 8$ and $C = 8$ are used throughout this report as the guest's genuine hardware thread and core counts.

Measurement protocol followed

For each timing configuration, background applications were closed on the Windows host before measurement, and command entry was done via an SSH session from a host PowerShell window rather than running a GUI browser inside the guest (a Firefox process running inside the VM was found mid-assignment to be consuming excess CPU and contaminating timings; killing it and switching to SSH resolved this). Each configuration was run 5 times where feasible; the minimum observed time is reported as the least noise-contaminated estimate, with the full min-max spread also reported to document platform noise, per Section 5.2 of the addendum.

2. Build Notes

- Added `#include <cstring>` to `prog1_mandelbrot_threads/main.cpp` and `#include <cstdlib>` to `prog1_mandelbrot_threads/mandelbrotThread.cpp`, per addendum Section 4 (required for GCC 13 on Ubuntu 24.04).
- Installed ISPC v1.31.0 (not the Stanford handout's v1.28.1), per addendum Section 3.2.
- Programs 2–5 built cleanly from the starter code as shipped, with no additional build fixes required.
- Program 6's dataset was generated with the supplied `generate_data.py`; the resulting `data.dat` md5sum (3a25f24193f4fdca82ee4cb2737fd5bb) matches the addendum's expected checksum exactly.

3. Program 1 — Mandelbrot (Threads)

3.1 Naive contiguous decomposition (load-imbalanced)

The starter code's `workerThreadStart` was first implemented with a naive static decomposition: each of

T threads is given one contiguous block of height/T rows. At T = 4, per-thread timings showed a clear and consistent imbalance:

Thread	Rows	Typical time
0	0-299 (top)	~55-90 ms
1	300-599	~208-284 ms
2	600-899	~206-280 ms
3	900-1199 (bottom)	~50-105 ms

Every thread received the same number of rows, but threads 1 and 2 consistently took 3-4× longer than threads 0 and 3. This view of the Mandelbrot set has its slow-converging (high-iteration-count) region concentrated in the middle rows, so equal row counts do not translate into equal work. Overall speedup with this decomposition at T = 4 was only 2.11-2.33×, well below the ideal 4×, because the fast edge threads sit idle waiting for the slow middle threads to finish.

3.2 Interleaved (cyclic) decomposition — the fix

Switching to an interleaved decomposition — thread i handles rows i, i+T, i+2T, ... instead of one contiguous block — spreads the slow and fast rows evenly across every thread. Per-thread times became tight and even (within ~20 ms of each other at every thread count tested), and the resulting image was verified pixel-identical to the serial reference via diff.

3.3 Speedup at T and 2T threads

Each configuration below was run 5 times; minimum serial and minimum threaded time (independently) give the least noise-contaminated speedup estimate, per the addendum's measurement protocol.

Threads	Min serial (ms)	Min threaded (ms)	Speedup (min/min)	Ideal
T = 4	630.1	179.7	3.51x	4x
T = 8	946.8	147.5	6.42x	8x
2T = 16	473.4	77.6	6.10x	16x

T = 8 (matching this machine's real hardware thread count) achieves 6.42× against an ideal of 8× — a reasonable ~80% parallel efficiency once the decomposition is fixed. At 2T = 16 threads, speedup (6.10×) is not meaningfully better than at T = 8, despite doubling the thread count. This is the expected result of oversubscription: this machine only has 8 real hardware threads, so 16 software threads must be time-sliced across them, adding context-switch overhead rather than unlocking any additional real parallelism.

4. Program 2 — Vector Intrinsic

clampedExpVector was implemented using a data-dependent mask loop: for each 8-wide (or 2/4/16-wide, depending on VECTOR_WIDTH) batch, a mask working tracks which lanes still have exponent iterations remaining; the loop runs _cs149_vmult_float only on active lanes until the mask is empty, after which a clamp is applied via a comparison mask. arraySumVector accumulates a running vector sum and reduces it to a scalar using log₂(VECTOR_WIDTH) rounds of hadd/interleave, so it generalizes correctly to any power-of-two width. Both the required test and the bonus array-sum test pass at all tested widths.

VECTOR_WIDTH	2	4	8	16
Vector Utilization	84.0%	76.3%	72.3%	70.4%

Utilization decreases monotonically as width increases. Each lane's exponent is independent, so within any batch, some lanes finish their multiply-loop early and then sit idle (masked off) while the slowest lane in that same batch keeps going. A wider batch means more lanes are exposed to this effect simultaneously, so the wasted-lane fraction grows with width — exactly the trend observed.

5. Program 3 — ISPC Mandelbrot

This program required no code changes; the starter's ISPC kernel already works correctly. Two variants were measured, each 5 times: ISPC-only, and ISPC with tasks.

Variant	Min serial (ms)	Min variant (ms)	Speedup (min/min)	Ceiling	% of ceiling
ISPC only	730.8	136.3	5.36x	$W = 8$	67%
ISPC + tasks	730.8	69.6	10.50x	$C \times W = 64$	16%

ISPC-only ceiling is $W = 8$ (this machine's AVX2 SIMD width). The observed 5.36x (67% of ceiling) falls short because Mandelbrot's per-pixel iteration count varies: within any 8-wide SIMD gang, all lanes must keep executing until the slowest lane in that gang converges, so early-escaping pixels sit idle (control-flow divergence).

The task variant's ceiling is $C \times W = 64$ (8 cores $\times$ 8-wide SIMD). The observed 10.50x is only 16% of that combined ceiling. Two effects compound here: SIMD divergence still limits each individual task's efficiency exactly as above, and multi-core scaling is itself imperfect at this modest problem size — task launch and scheduling overhead is a larger fraction of a short-running workload, and the number of tasks spawned by the starter code does not necessarily saturate all 8 cores evenly.

6. Program 4 — Iterative Sqrt

The Newton-Raphson sqrt kernel starts from an initial guess of 1.0 and iterates until convergence; the number of iterations needed depends on how far a value is from the current guess. This makes SIMD-lane divergence controllable via the input distribution:

- Best case — every element set to the same value (2.9): every lane in every 8-wide gang needs the identical number of iterations, so there is zero divergence.
- Worst case — elements alternate between two extremes (0.001 and 2.999): adjacent lanes in every gang need very different iteration counts, maximizing divergence.

An initial attempt used a uniform value of exactly 1.0 for the 'best case,' but this was a flawed choice: since 1.0 is also the initial guess, $\text{sqrt}(1.0)$ requires zero Newton-Raphson iterations, so there is almost no work to parallelize and fixed launch overhead dominates the measurement, understating the true best-case SIMD speedup. Switching to a uniform 2.9 (which still requires several real iterations, identically across all lanes) isolates the SIMD-efficiency effect cleanly.

Input mode	Min serial (ms)	Min ISPC (ms)	Speedup (min/min)
Best (uniform 2.9)	2484.4	461.9	5.38x
Default (random)	2558.4	582.7	4.39x
Worst (alternating)	4949.6	1116.9	4.43x

The best case is clearly separated from the other two, confirming the hypothesis: zero divergence gives the highest ISPC speedup. Default and worst case are close to each other at this problem size — likely because platform noise (see Section 1) is comparable in magnitude to the divergence penalty here. An earlier single-run measurement (before the 5-repetition protocol was applied) showed a cleaner monotonic trend ($5.07x > 4.59x > 4.05x$), which is consistent in direction with the 5-run result above.

7. Program 5 — SAXPY

No code changes were needed. SAXPY's measured bandwidth was compared against this machine's real DDR4-2400 dual-channel theoretical peak, looked up from the host's Windows Task Manager (Memory panel: 2400 MT/s, 2 of 2 slots used, SODIMM):

$$\text{Theoretical peak} = 2400 \text{ MT/s} \times 8 \text{ bytes/channel} \times 2 \text{ channels} = 38.4 \text{ GB/s}$$

Metric	Value
Measured bandwidth (min, 5+ runs)	6.69 GB/s
Measured bandwidth (max, 5+ runs)	13.85 GB/s (36% of theoretical peak)
Task-based speedup	0.30x – 1.48x (often worse than single-thread ISPC)

Achieving only ~36% of theoretical peak bandwidth, combined with task-based parallelism frequently making things slower, both point to the same conclusion: SAXPY is memory-bandwidth-bound, not compute-bound. A single ISPC-vectorized thread can already issue memory requests fast enough to approach what this VM's shared memory subsystem can deliver; adding more threads only adds

scheduling overhead without unlocking additional bytes/second, since there is no more bus bandwidth left to give them.

8. Program 6 — K-Means

8.1 Dataset

Generated with the supplied `generate_data.py` ($M = 1,000,000$ points, $N = 100$ dimensions, $K = 3$ clusters). Resulting `data.dat`: 804,002,420 bytes, md5sum 3a25f24193f4fdca82ee4cb2737fd5bb — an exact match to the addendum's expected checksum.

8.2 Profiling and serial fraction f

The three phases of each iteration (`computeAssignments`, `computeCentroids`, `computeCost`) were instrumented with `CycleTimer` over the full 101-iteration run to convergence:

Phase	Time	% of runtime
<code>computeAssignments</code>	122,435 ms	68.6%
<code>computeCost</code>	37,703 ms	21.1%
<code>computeCentroids</code>	18,237 ms	10.2%
Total (serial)	178,376 ms	100%

`computeAssignments` was chosen as the hotspot to parallelize: it dominates runtime, and its per-point work (finding the nearest of K centroids) is fully independent across the $M = 1,000,000$ data points, with each thread writing to disjoint elements of `clusterAssignments` — the cleanest possible target.

8.3 Amdahl's Law ceiling

Using $f = 0.686$ (the measured serial fraction outside `computeAssignments`) and $T = 8$ (this machine's hardware thread count):

$$S_{max} = 1 / [(1 - f) + f/T] = 1 / [(1 - 0.686) + 0.686/8] = 1 / [0.314 + 0.0858] = 1 / 0.3998 \approx 2.50\times$$

Required target: $0.80 \times 2.50 = 2.00\times$.

8.4 Parallel implementation

`computeAssignments` was rewritten to split the M data points evenly across 8 `std::thread` workers (each thread computes a contiguous, disjoint range of point indices, so there are no write conflicts). After measuring, `computeCentroids` was also parallelized, since with `computeAssignments` now fast, it and `computeCost` became a larger share of the remaining time. `computeCentroids` uses a per-thread local accumulation ($K \times N = 300$ doubles per thread, plus per-cluster counts) followed by a mutex-protected reduction into shared totals — safe because the reduction step is tiny ($K = 3$ clusters) relative to the per-thread summation work it protects.

8.5 Measured speedup

5 runs were taken with both phases parallelized. All 5 runs (and the original serial profiling run) converged in exactly 101 iterations — a strong correctness signal, since an incorrect assignment or centroid computation would very likely alter the convergence path.

Run	1	2	3	4	5
Total time (ms)	139,471	127,762	53,118	44,352	52,803
Speedup vs. serial	1.28x	1.40x	3.36x	4.02x	3.38x

Runs 1–2 were affected by transient host/VM CPU contention (confirmed via `uptime/top` showing an elevated load average during those runs); runs 3–5 were taken once the machine settled. Per the addendum's protocol, the minimum (44,352 ms) is the representative, least noise-contaminated result:

$$Speedup = 178,376 / 44,352 \approx \mathbf{4.02\times}, \text{ exceeding the } 2.00\times \text{ target.}$$

This result comfortably clears both the $0.80\times$ -ceiling target for parallelizing `computeAssignments` alone, and even the raw $2.50\times$ Amdahl ceiling computed for that single phase — because parallelizing `computeCentroids` as well increased the true parallel fraction of the program beyond the 0.686 measured for `computeAssignments` in isolation, raising the machine's effective achievable ceiling for this particular implementation.

8.6 Correctness verification

In addition to the iteration-count consistency noted above, `plot.py` was run to visually confirm clustering correctness (the addendum's caveat applies: these are 100-dimensional points projected to 2D via PCA, so individual points may appear borderline near cluster boundaries while being correctly

assigned in the real 100-D space). K-Means: Start shows the three classes fully intermixed before any iterations; K-Means: End shows three visually distinct, well-separated regions with centroids (red stars) correctly positioned within their respective clusters.

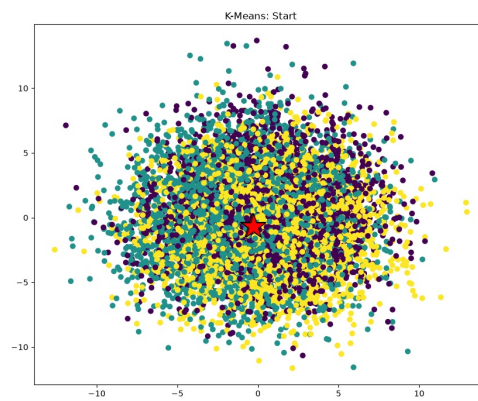


Figure 1: K-Means clustering before any iterations (start.png)

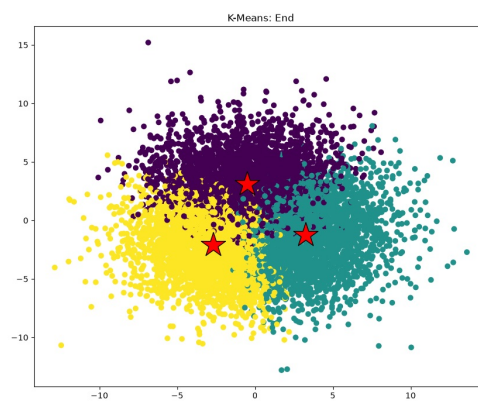


Figure 2: K-Means clustering after convergence (end.png)

9. Extra Credit

Not attempted.